

BUSN 41210: Financial Analytics

Lecture 5: Regularization for Linear Models: Ridge & Lasso

Dacheng Xiu

Chicago Booth

September 16, 2026

© 2026 Dacheng Xiu, Chicago Booth Restricted Materials. All Rights Reserved.

Outline

- ▶ Maximum Likelihood Estimation
- ▶ Regularization
 - ▶ Ridge
 - ▶ Lasso
 - ▶ Elastic Net
- ▶ Cross Validation
- ▶ Design an OOS Experiment
 - ▶ Splitting Sample
 - ▶ Covariates Standardization
 - ▶ Choice of Parameter Grid

Maximum Likelihood Estimation

Two related concepts:

Likelihood is a function of the unknown parameters β of a statistical model, given data:

$$\text{LHD}(\beta) = P(\text{data}|\beta)$$

Maximum Likelihood (ML) Estimation:

$$\hat{\beta} = \underset{\beta}{\operatorname{argmax}} \text{LHD}(\beta)$$

ML estimates $\hat{\beta}$ are those parameter values β that are most likely to have generated our data.

Minimizing Deviance

Two related concepts:

Deviance refers to the distance between data and fit.

You want to make it as small as possible.

$$Dev(\beta) = -2\log \text{LHD}(\beta) + C$$

C is a constant you can mostly ignore.

Deviance is useful for comparing models.

Deviance is a measure of GOF that plays the role of residual sums of squares for a broader class of models (logistic regression etc.)

We'll think of deviance as a cost to be minimized.

Minimize deviance $\Leftrightarrow$ maximize likelihood.

Least-Squares and Deviance in Linear Regression

The probability model is $y|\mathbf{x} \sim N(\mathbf{x}'\boldsymbol{\beta}, \sigma^2)$.

density of $N(\mu, \sigma^2) = \exp \left[-(y - \mu)^2 / 2\sigma^2 \right] / \sqrt{2\pi\sigma^2}$.

Given n independent observations, the likelihood is

$$\prod_{i=1}^n \text{pr}(y_i|\mathbf{x}_i) \propto \exp \left[-\frac{1}{2} \sum_{i=1}^n (y_i - \mathbf{x}_i'\boldsymbol{\beta})^2 / \sigma^2 \right]$$

This leads to $Dev \propto \frac{1}{\sigma^2} \sum_{i=1}^n (y_i - \mathbf{x}_i'\boldsymbol{\beta})^2$.

Minimizing deviance is the same as least squares!

And thus the MLE minimizes our sum of squared errors.

Recall: Variance of OLS Estimates

- ▶ OLS estimates have no bias:

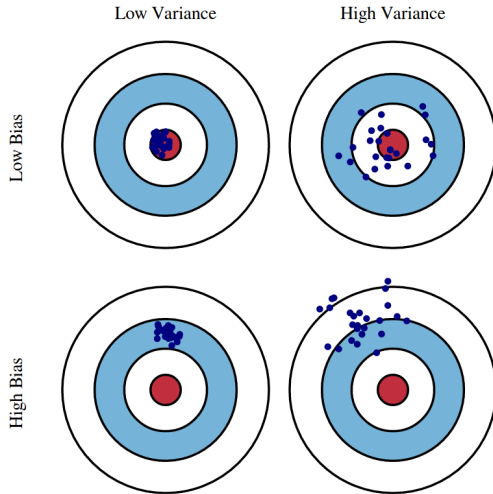
$$E(\hat{\beta}_{\text{OLS}}) = \beta.$$

- ▶ OLS estimates may have large variance:

$$\text{Var}(\hat{\beta}_{\text{OLS}}) = \sigma^2(X'X)^{-1},$$

as the number of predictors p approaches or exceeds n , that the matrix $X'X$ becomes closer to singular or poorly conditioned.

Bias-Variance Tradeoff



Regularization

The general idea behind regularization:

impose restrictions on the MLE estimates such that they are suitably disciplined (stable and purposeful).

What do we mean by 'suitable restrictions'?

- ▶ We would like our estimates $\hat{\beta}$ to be less variable,
- ▶ We would like our estimates $\hat{\beta}$ to yield better prediction,
- ▶ We would like our estimates $\hat{\beta}$ to have exact zeros.

We will **penalize** solutions β that are not desirable. Ultimately, we are departing from optimality to stabilize the system (bias-variance tradeoff).

Regularization

Using all predictors $x_1, \dots, x_p$, the *maximum likelihood estimator* $\hat{\beta}_{MLE}$ minimizes deviance:

$$\hat{\beta}_{MLE} = \underset{\beta}{\operatorname{argmin}} -\frac{2}{n} \log \text{LHD}(\beta)$$

$\hat{\beta}_{MLE}$ can only be obtained when $p < n$, it can be unstable and it cannot achieve exact zeros.

The penalized maximum likelihood estimator $\hat{\beta}_{PMLE}$ minimizes deviance **plus a penalty**:

$$\hat{\beta}_{PMLE} = \underset{\beta}{\operatorname{argmin}} -\frac{2}{n} \log \text{LHD}(\beta) + \text{pen}(\beta)$$

Penalty incurs **cost** when choosing solutions that are far from our preferences.

Decision Theory: Cost in Estimation

Decision theory is about how to behave optimally under uncertainty. It is based on the idea that choices have costs.

Estimation and hypothesis testing: what are the costs?

Estimation:

Deviance is the cost of distance between data and the model.

Testing:

Since $\hat{\beta}_j = 0$ is *safe*, it should cost us to decide otherwise.

$\Rightarrow$ The cost of $\hat{\beta}$ is deviance plus a penalty away from zero.

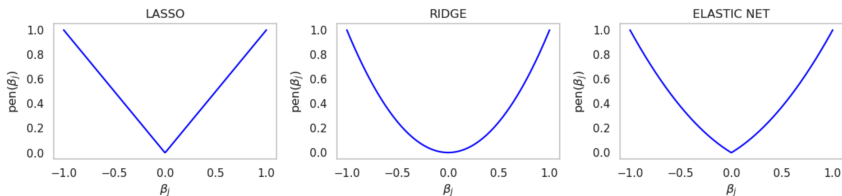
Regularized Regression

Sparsity-inducing penalization:

$$\hat{\beta}_{PMLE} = \underset{\beta}{\operatorname{argmin}} \left\{ -\frac{2}{n} \log \text{LHD}(\beta) + \lambda \sum_j \text{pen}(\beta_j) \right\}$$

$\lambda > 0$ is the penalty weight, pen is a cost (penalty) function.

$\text{pen}(\beta)$ will be lowest at $\beta = 0$ and we pay more for $|\beta| > 0$.



Options: ridge β^2 , lasso $|\beta|$, elastic net $\alpha \beta^2 + (1 - \alpha) |\beta|$, ...

Ridge Regression

- Penalty added to the least squares objective:

$$\hat{\beta}_{Ridge}(\lambda) = \arg \min_{\beta} \sum_{i=1}^n \left(y_i - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p \beta_j^2$$

- $\lambda \geq 0$ is complexity parameter controlling shrinkage: higher $\lambda \leftrightarrow$ more shrinkage
- Equivalent problem makes the magnitude constraint explicit:

$$\hat{\beta}_{Ridge}(\tau) = \arg \min_{\beta} \sum_{i=1}^n \left(y_i - \sum_{j=1}^p \beta_j x_{ij} \right)^2 \quad \text{s.t.} \quad \sum_{i=1}^p \beta_i^2 \leq \tau$$

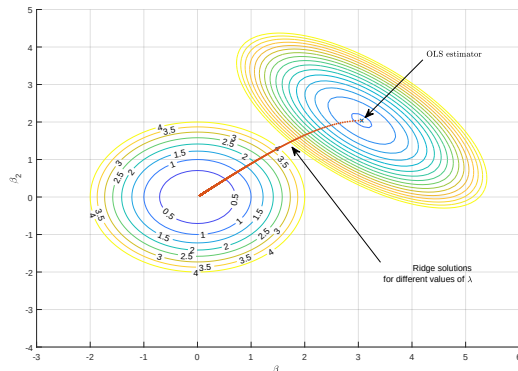
- The solution is given in closed-form:

$$\hat{\beta}_{Ridge}(\lambda) = (\mathbf{X}'\mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}'\mathbf{Y}.$$

*Other names: Tikhonov regularization, weight decay, L^2 regularization

Ridge As A Constrained Optimization Program

- The figure represents the sum-of-squared errors level curves as well as the Ridge restrictions.
- The cross indicates the OLS (unrestricted) solution.
- Red dots represent the Ridge solutions as a function of λ .



Why does Ridge “Work” ?

- The estimator is OLS on an **augmented dataset**:

$$\mathbf{X}_\lambda = \begin{pmatrix} x_{11} & x_{12} & x_{13} & \cdots & x_{1p} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ x_{n1} & x_{n2} & x_{n3} & \cdots & x_{np} \\ \sqrt{\lambda} & 0 & 0 & \cdots & 0 \\ 0 & \sqrt{\lambda} & 0 & \cdots & 0 \\ 0 & 0 & \sqrt{\lambda} & \ddots & 0 \\ 0 & 0 & 0 & \ddots & 0 \\ 0 & 0 & 0 & \cdots & \sqrt{\lambda} \end{pmatrix}; \mathbf{Y}_\lambda = \begin{pmatrix} y_1 \\ \vdots \\ y_n \\ 0 \\ 0 \\ 0 \\ \vdots \\ 0 \end{pmatrix}$$

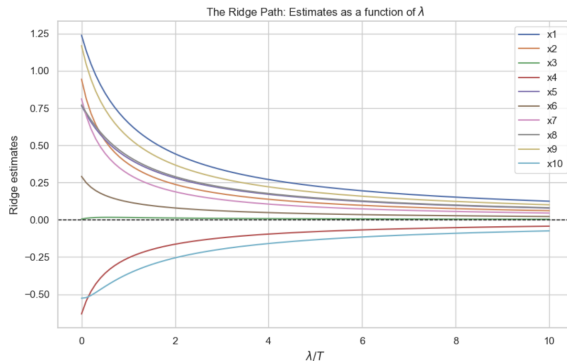
Why does Ridge “Work” ?

- Hence, we have

$$\begin{aligned}\hat{\beta}_{Ridge}(\lambda) &= (\mathbf{X}'_{\lambda} \mathbf{X}_{\lambda})^{-1} \mathbf{X}'_{\lambda} \mathbf{Y}_{\lambda} \\ &= \left[(\mathbf{X}', \sqrt{\lambda} \mathbf{I}) \begin{pmatrix} \mathbf{X} \\ \sqrt{\lambda} \mathbf{I} \end{pmatrix} \right]^{-1} (\mathbf{X}', \sqrt{\lambda} \mathbf{I}) \begin{pmatrix} \mathbf{Y} \\ \mathbf{0} \end{pmatrix} \\ &= (\mathbf{X}' \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}' \mathbf{Y}.\end{aligned}$$

- $\mathbf{X}'_{\lambda} \mathbf{X}_{\lambda}$ will not be singular even if $\mathbf{X}' \mathbf{X}$ is singular.

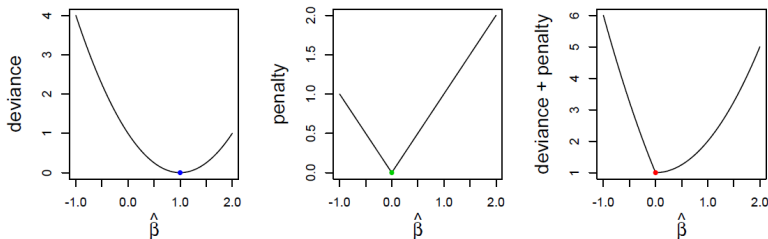
The Ridge Path: Estimates As A Function of λ



- ▶ Simulated data from a linear regression model with $p = 10$ covariates. Number of observations: $T = 100$.
- ▶ The figure shows the Ridge estimates as a function of λ .
- ▶ FACT: The ridge path is continuous, and the estimates go to zero only as $\lambda \rightarrow \infty$.

Penalization and Automatic Variable Selection

The minimum of a smooth + spiky function can be at the spike.



The PMLE estimator is **shrunk to zero** $\Rightarrow$ automatic variable selection

For instance, LASSO has this property.

You can think of the LASSO as the *modern least squares*.

LASSO

“least absolute shrinkage and selection operator”

$$\hat{\beta}_{\text{LASSO}}(\lambda) = \arg \min_{\beta} \sum_{i=1}^n \left(y_i - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p |\beta_j| \quad (\text{or, } \min \text{MSE s.t. } \sum_{j=1}^p |\beta_j| \leq \tau)$$

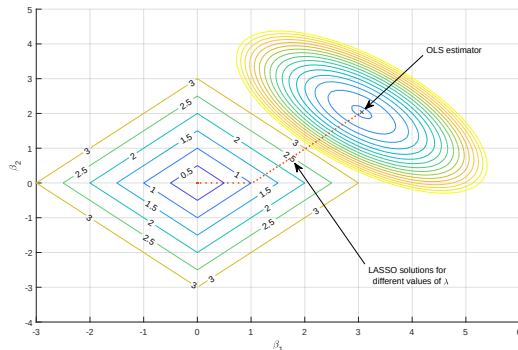
Unlike ridge

- ▶ No closed-form solution in general (but efficient algorithms place it at roughly same computational cost)
- ▶ For sufficiently large λ , **some coefficient estimates will be exact zeros**
- ▶ Thus lasso a **model selection** alternative to stepwise regression

*Other names: L^1 regularization, basis pursuit

LASSO As A Constrained Optimization Program

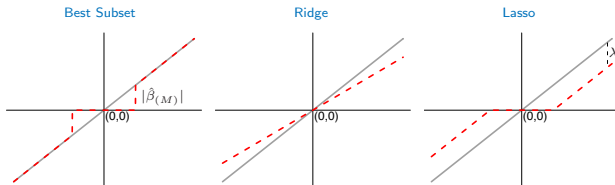
- The figure represents the sum-of-squared errors level curves as well as the LASSO restrictions.
- The cross indicates the OLS (unrestricted) solution.
- Red dots represent the LASSO solutions as a function of λ .



LASSO, Ridge, Subset

For $X'X = \mathbf{I}$	
Estimator	Formula
Best subset (size K)	$\hat{\beta}_j \mathbf{1}_{ \hat{\beta}_j^{ols} \geq \hat{\beta}_{(K)}^{ols} }$
Ridge	$\hat{\beta}_j^{ols} / (1 + \lambda)$
Lasso	$\text{sign}(\hat{\beta}_j^{ols})(\hat{\beta}_j^{ols} - \lambda)^+$

- ▶ Best subset drops all variables with coefficients smaller than the K th largest (“hard thresholding”)
- ▶ Ridge regression does a proportional shrinkage
- ▶ Lasso translates each coefficient by constant factor λ , truncating at zero (“soft thresholding”)



LASSO Paths

The lasso fits $\hat{\beta}$ to minimize $-\frac{2}{n} \log \text{LHD}(\beta) + \lambda \sum_j |\beta_j|$. How to pick λ ?

We'll do this for a *sequence* of penalties $\lambda_1 > \lambda_2 \dots > \lambda_T$.

Then we can apply model selection tools to choose best $\hat{\lambda}$.

Path estimation:

Start with big λ_1 so big that $\hat{\beta} = \mathbf{0}$.

For $t = 2 \dots T$: update $\hat{\beta}$ to be optimal under $\lambda_t < \lambda_{t-1}$.

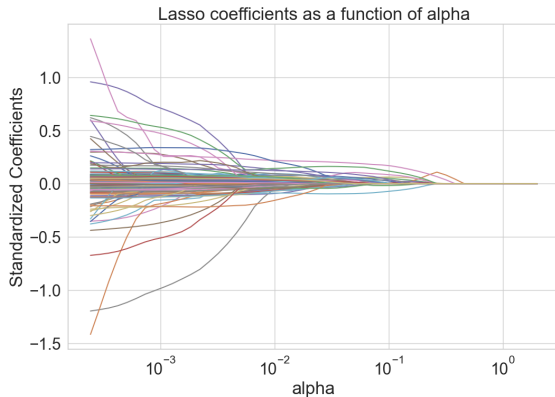
Since estimated $\hat{\beta}$ changes smoothly along this path:

- ▶ It's fast! Each update is easy.
- ▶ It's stable: optimal λ_t may change a bit from sample to sample, but that won't affect the model much.

It's a better version of forward stepwise selection.

Path Plots

The whole enterprise is easiest to understand visually.



The algorithm moves *right to left*.

The y-axis is standardized $\hat{\beta}$ (each line a different $\hat{\beta}_j$) as a function of λ_t .

Scale Matters

Penalization is sensitive to the scale of $x_1, \dots, x_p$. e.g., $x\beta$ has the same effect as $(2x)\beta/2$, but $|\beta|$ is twice as much penalty as $|\beta/2|$.

It would not be 'fair' to penalize β 's equally if x 's were not on the same scale.

You can multiply β_j by $\text{sd}(x_j)$ in the cost function to standardize.

That is, minimize $-\frac{2}{n} \log \text{LHD}(\beta) + \lambda \sum_j \text{sd}(x_j) |\beta_j|$.

$\Rightarrow \beta_j$'s penalty is calculated per effect of 1SD change in x_j .

How Much Regularization?

The LASSO minimizes $-\frac{2}{n} \log \text{LHD}(\beta) + \lambda \sum_j |\beta_j|$.

This ‘sparse regularization’ auto-selects the variables.

Sound too good to be true? You need to choose λ .

Think of $\lambda > 0$ as a signal-to-noise filter: *like squelch on a radio*.

We'll use **cross validation** or **information criteria** to choose.

Path algorithms are key to the whole framework:

- ★ They let us quickly enumerate a set of candidate models.
- ★ This set is stable, so selected ‘best’ is probably pretty good.

Prediction vs Interpretability

Model selection can be used to tackle two tasks:

- ▶ learning about the data-generating-mechanism. (what are the contributing variables and can we interpret them?)
- ▶ prediction.

None of your models will be 'true' for complicated HD systems.
Instead, just try to get as close to the truth as possible.

Parsimony principle: If two models do similarly well in predicting the unseen data, choose the simpler one.

- ▶ Overly simple models will 'underfit' available patterns.
- ▶ Complicated models 'overfit', and make noisy predictors.

The goal is to find the sweet spot in the middle.

Prediction-driven Model Selection: It is All About Prediction.

A recipe for model selection.

1. Find a manageable set of candidate models (i.e., such that fitting all models is fast).
2. Choose amongst these candidates the one with best predictive performance *on unseen data*.

1. is what the LASSO paths provide.
2. Seems impossible! But it's not ...

We need to *estimate* the prediction accuracy associated with each model when applied on future unseen data.

Recall that **predictive performance** can be measured with 'deviance'.

Considerations for Out-of-Sample Experiments

- ▶ Sample splitting
 - ▶ Large datasets: A traditional split (e.g., 80/20 or 70/30) is feasible and offers good generalization.
 - ▶ Small datasets: Splitting too much data into validation can hinder effective training.
- ▶ Temporal or Sequential Data
 - ▶ Ensure the order of data is respected to prevent leakage and maintain sequence integrity.
- ▶ Covariate Scaling
 - ▶ Be mindful of information leakage by scaling after splitting the data.
- ▶ Grid Selection for Tuning Parameters
 - ▶ Ensure the optimal tuning parameters fall within the middle of the chosen grid.

Sample Splitting for I.I.D. Data

Consider K -fold cross-validation for i.i.d. data.

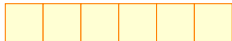
- ▶ This guarantees each observation is left-out for validation, and lowers the sampling variance of CV model selection.
- ▶ Leave-one-out CV, with $K = n$, is nice but takes a long time. $K = 5$ to 10 is fine in most applications.

K-Fold Cross-Validation



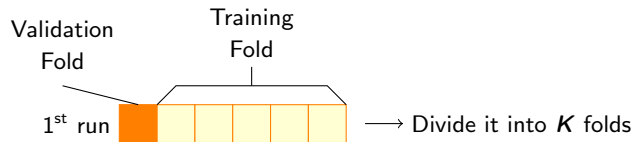
→ Take your dataset

***K*-Fold Cross-Validation**

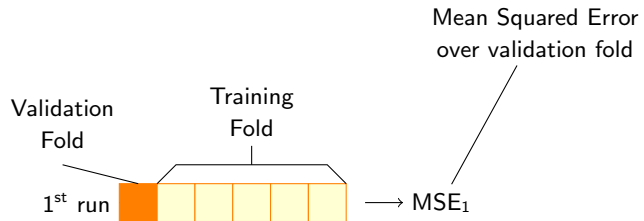


→ Divide it into K folds

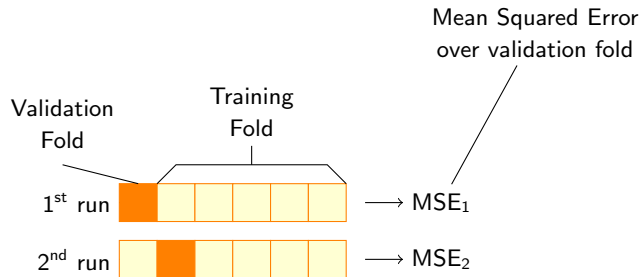
***K*-Fold Cross-Validation**



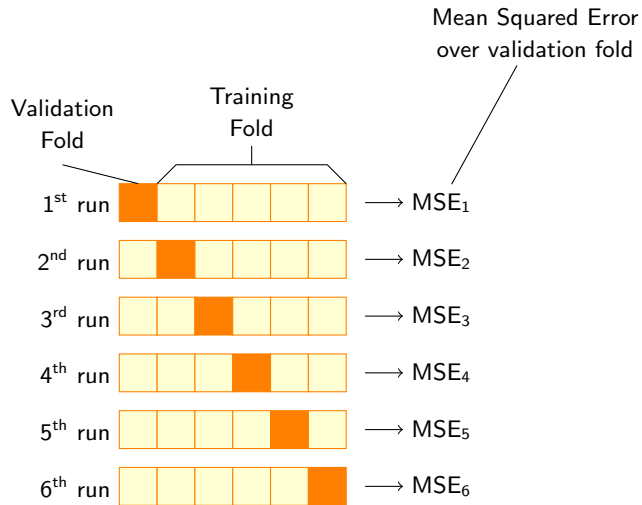
K-Fold Cross-Validation



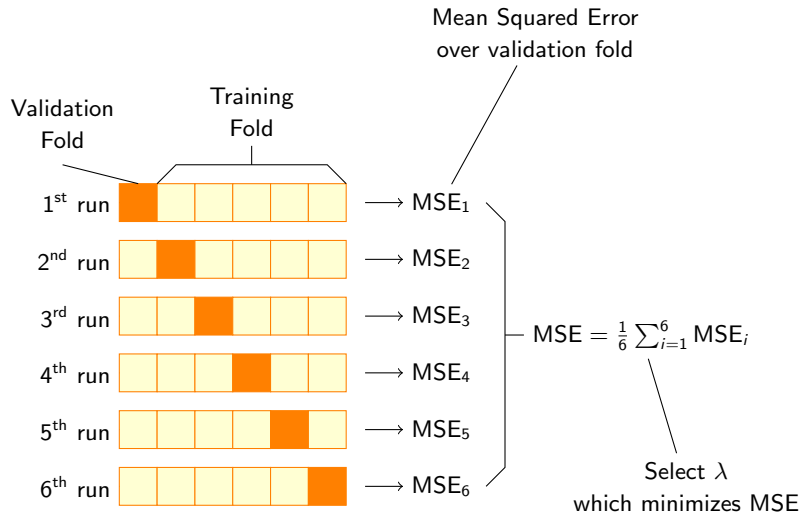
K-Fold Cross-Validation



K-Fold Cross-Validation



K-Fold Cross-Validation



CV LASSO

The lasso path algorithm minimizes $-\frac{2}{n} \log \text{LHD}(\beta) + \lambda_t \sum_j |\beta_j|$ over the sequence of penalty weights $\lambda_1 > \lambda_2 \dots > \lambda_T$.

This gives us a path of T fitted coefficient vectors, $\hat{\beta}_1 \dots \hat{\beta}_T$, each defining deviance for new data: $-\log p(\mathbf{y}^{\text{new}} \mid \mathbf{X}^{\text{new}} \hat{\beta}_t)$.

Set a sequence of penalties $\lambda_1 \dots \lambda_T$.

Then, for each of $k = 1 \dots K$ folds,

- ▶ Fit the path $\hat{\beta}_1^k \dots \hat{\beta}_T^k$ on all data *except* fold k .
- ▶ Get fitted deviance *on left-out data*: $-\log p(\mathbf{y}^k \mid \mathbf{X}^k \hat{\beta}_t)$.

This gives us K draws of OOS deviance for each λ_t .

Finally, use the results to choose the 'best' $\hat{\lambda}$, then re-fit the model to *all of the data* by minimizing $-\frac{2}{n} \log \text{LHD}(\beta) + \hat{\lambda} \sum_j |\beta_j|$.

Macro Forecasting Example: CV LASSO

```
> alphas=2**(np.linspace(-12,1,50))  
> K = 10  
> model_CV = LassoCV(alphas=alphas, cv=K)  
> model_CV.fit(X_train_scaled, y_train_scaled)
```

Optimal Alpha: 0.06075

Number of Variables: 22

LassoCV IS R2: 0.35

LassoCV OOS R2: -0.03

- ▶ Selecting the proper range for the tuning parameter is important.
- ▶ If the tuning parameter is too large, the model might effectively shrink most coefficients to nearly zero, making the variables insignificant.
- ▶ If the tuning parameter is too small, the model will include all variables with little to no penalty.

Macro Forecasting Example: CV LASSO

► With Large λ :

```
> alphas_large=10**(np.linspace(0,10,50))  
> K = 10  
> model_CV_large = LassoCV(alphas=alphas_large, cv=K)  
> model_CV_large.fit(X_train_scaled, y_train_scaled)  
> alpha_CV_large = model_CV_large.alpha_
```

```
alpha_CV_large: 100000000000.0
```

```
Number of Variables: 0
```

```
LassoCV IS R2: 0.0
```

```
LassoCV OOS R2: 0.0
```

Macro Forecasting Example: CV LASSO

► With Small λ :

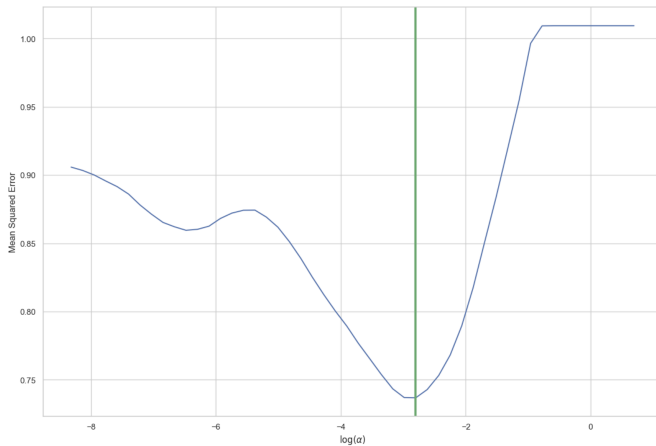
```
> alphas_small=10**(np.linspace(-20,-10,50))  
> K = 10  
> model_CV_small = LassoCV(alphas=alphas_small, cv=K)  
> model_CV_small.fit(X_train_scaled, y_train_scaled)  
> alpha_CV_small = model_CV_small.alpha_
```

```
alpha_CV_small: 1e-10  
Number of Variables: 119  
LassoCV IS R2: 0.52  
LassoCV OOS R2: -0.4
```

Macro Forecasting Example: CV LASSO

► With Optimal λ :

Again, the routine is most easily understood visually.



Problems with Cross Validation

It is time consuming: When estimation is not instant, fitting K times can become unfeasible even K in 5-10.

It can be unstable: imagine doing CV on many different samples. There can be large variability on the model chosen.

Still, some form of CV is used in most DM applications.

Also, be careful to not cheat: for example, with the screening cut model we've already used the full n observations to select the strongest variables. It is not surprising they do well 'OOS'.

The rules: never use the same data twice (for selection and validation).

(Alternatives to CV) Information Criteria

'Information Criteria' (IC): measure how much information is lost when we use our model to represent our data. They approximate distance between a model and 'the ideal'.

IC provide an avenue for **in-sample model comparisons**, by trading off *in-sample* deviance and *model complexity*.

You can apply them by choosing the model with minimum IC.

Most common is Akaike's $AIC = IS\ Deviance + 2df$.

df = 'degrees of freedom' used in your model fit.

For lasso and MLE, this is just the # of nonzero $\hat{\beta}_j$.

AIC Overfits in High Dimensions

The AIC is actually estimating OOS deviance: what your deviance would be on another *independent* sample of size n .

IS deviance is too small, since the model is tuned to this data.

Some deep theory shows that OOS - IS deviance $\approx 2df$.

$$\Rightarrow \text{AIC} \approx \text{OOS deviance}.$$

It's common to claim this approx (i.e., AIC) is good for 'big n '.

Actually, its only good for big n/df .

In Big Data, df (# parameters) can be huge. Often $df \approx n$.

In this case the AIC will be a bad approximation: it overfits!

(Another option) Bayes IC

The BIC is $\text{IS Deviance} + \log(n) \times df$.

This *looks* just like AIC, but comes from a very different place.

$BIC \approx -\log p(M_b|\text{data})$, the ‘probability that model b is true’.

$$p(M_b|\text{data}) = \frac{p(\text{data}, M_b)}{p(\text{data})} \propto \underbrace{p(\text{data}|M_b)}_{\text{LHD}} \underbrace{p(M_b)}_{\text{prior}}$$

The ‘prior’ is your probability that a model is true *before* you saw any data. BIC uses a ‘unit-info’ prior: $N\left[\hat{\beta}, \frac{2}{n}\text{var}(\hat{\beta})^{-1}\right]$

AIC tries to approx OOS deviance.

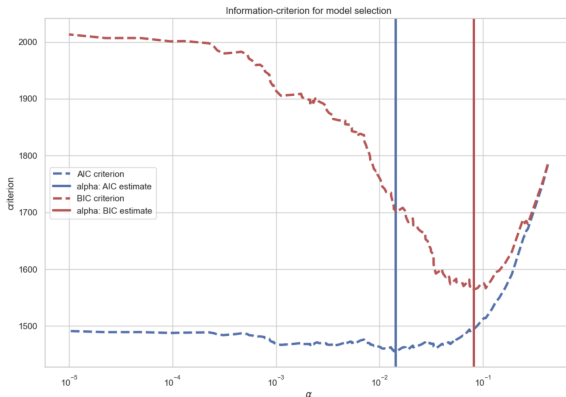
BIC is trying to get at the ‘truth’.

Macro Forecasting Example: AIC vs BIC

In fact, our macroeconomic data has 743 observations and 119 variables. It is difficult to claim that we have a big n/df .

```
> X1.shape
```

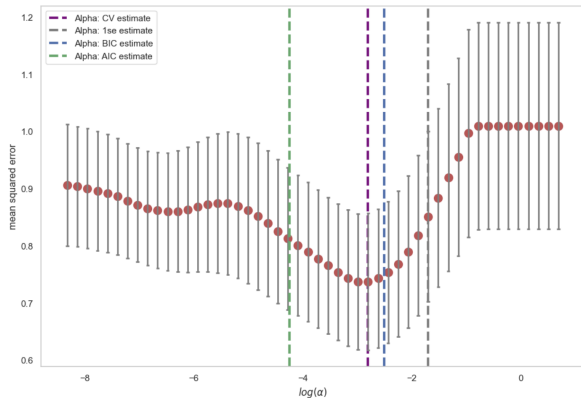
```
Output: (743, 119)
```



	AIC	BIC
#variables	57.00	17.00
R2	0.44	0.32
OOS R2	-0.22	-0.01

IC and CV on the Macroeconomics Data

```
> model_aic = LassoLarsIC(criterion='aic').fit(X_train_scaled, y_train_scaled)
> model_bic = LassoLarsIC(criterion='bic').fit(X_train_scaled, y_train_scaled)
```



	AIC	BIC	CV
#variables	57.00	17.00	22.00
R2	0.44	0.32	0.35
OOS R2	-0.22	-0.01	-0.03

LassoCV implemented with Lasso Path

LassoCV() shown above has an issue when it comes to scaling..

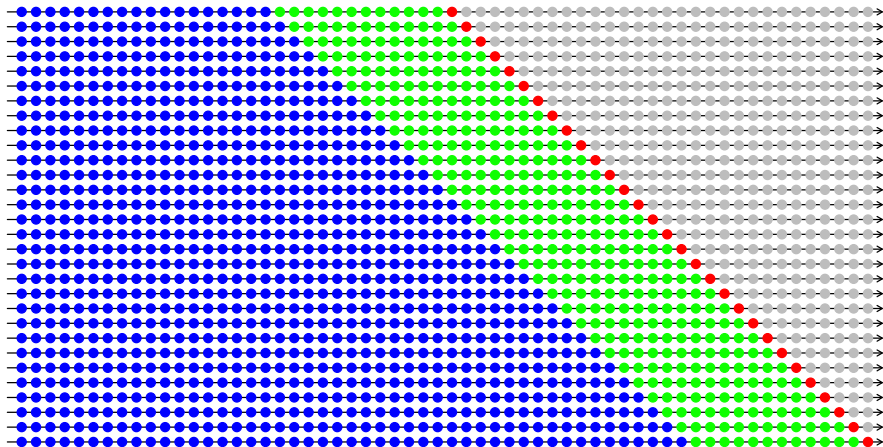
- ▶ The correct way to perform cross-validation (CV) is to scale using only the training data.
- ▶ In the LassoCV() example above, both the training and validation sets were scaled, which is not ideal.
- ▶ To resolve this issue, you should implement LassoCV using Lasso Path.

Cross-Validation for Time-Series Data

- ▶ Usually, CV is not valid with time-series data.
- ▶ Several solutions can be valid to solve this issue.
 - ▶ Expanding Window CV
 - ▶ The expanding window scheme is a training and validating approach where the training dataset starts with a small subset of the data and incrementally incorporates more data points over time.
 - ▶ This results in the training dataset growing to encompass a larger portion of the historical data, while the validation set advances along the time axis, ensuring that the models are trained on increasingly comprehensive historical data.
 - ▶ Rolling Window CV
 - ▶ The rolling window scheme involves advancing both the training and validation sets forward in time by a fixed interval.
 - ▶ This strategy ensures that the models are consistently trained and validated on consecutive datasets that are regularly updated with new data, maintaining a constant window size for both training and testing periods.

Recursive Time-ordered Validation and Testing

Training (Blue), Validation (Green), Testing (Red)



Expanding Window Algorithm

```
# generate expanding window iterator
> def expanding_window_iterator(data, initial_train_size, validation_size, test_size ,step):
    tscv_expanding = []
    train_size = initial_train_size
    for i in range(0,len(data) - train_size - validation_size - test_size +1,step):
        train_index = data[:i+train_size].index.values.astype(int)
        tscv_expanding.append(train_index)
    return tscv_expanding
```

- ▶ This algorithm starts with a **fixed starting point** and expands the training set by adding new data points in each iteration.
- ▶ It is important to maintain the same starting point in each expanding window to ensure that the model is trained consistently over time.
- ▶ By doing this, we ensure that the model learns incrementally from all available past data, leading to more reliable predictions.

Macro Forecasting Example: Expanding Window Algorithm

```
> initial_train_size = 180 # 15 years  
> validation_size = 84 # 7 years  
> test_size = 12 # 1 year  
> step = 12 # window size that expands training data: 1 year
```

- ▶ Before you run the algorithm, you need to set the parameters of "expanding_window_iterator" function.
- ▶ By setting "initial_train_size", you can set the training data size of the first iteration.
- ▶ The "validation_size" and the "test_size" are the fixed values to choose the validation data size and testing data size for each window.
- ▶ "step" is the window size that training data incremental amount for each iteration.

Macro Forecasting Example: Expanding Window Algorithm (Lasso)

```
> alphas, coefs, _ = lasso_path(X_train_scaled, y_train_scaled, alphas=alphas)
> coefs = coefs[0]
```

```
> def predict(Xs, coef):                                # scale in, y units out
    return yscaler.inverse_transform(
        (Xs @ coef).reshape(-1, 1)).reshape(-1)
```

```
> for idx, alpha in enumerate(alphas):
    coef = coefs[:, idx]
    dev = deviance(y_valid.values, predict( X_valid_scaled, coef),
                  family = 'gaussian')
    dev_ser.loc[alpha] = dev
    mse.loc[alpha] = dev / len(y_valid)
    best_pred[alpha] = predict( X_test_scaled, coef)
```

```
> alpha_best = alphas[np.argmin(mse.values)]
```

- For each window, we obtain the optimal alpha that minimizes MSE (with validation dataset).
- Note: "y_pred" from the testing dataset has nothing to do with obtaining optimal alpha.

Macro Forecasting Example: Expanding Window Algorithm (Lasso)

```
expanding_list = expanding_window_iterator(y_full, initial_train_size, validation_size, test_size, step)
alphas = 2 ** (np.linspace(-10, -1.3, 50))
table_mse, table_alpha, table_model, table_n_features, table_r2, table_pred = lasso_solver(X_full, y_full,
                                                                                       expanding_list, validation_size, test_size, params=alphas)
```

table_mse:

	0	1	2	3	4	5	6	7	8	9	...
Lasso_mse	0.006786	0.007052	0.007505	0.007412	0.006714	0.006503	0.004617	0.003907	0.00265	0.002328	...
29	30	31	32	33	34	35	36	37	38		
	0.007371	0.007338	0.006761	0.006742	0.006764	0.003188	0.001829	0.001809	0.001884	0.002173	

table_alpha:

	0	1	2	3	4	5	6	7	8	9	...
Alpha_best	0.406126	0.406126	0.406126	0.359098	0.317515	0.317515	0.406126	0.406126	0.406126	0.406126	...
29	30	31	32	33	34	35	36	37	38		
	0.317515	0.317515	0.317515	0.317515	0.317515	0.406126	0.406126	0.406126	0.406126	0.406126	

table_n_features:

	0	1	2	3	4	5	6	7	8	9	...	29	30	31	32	33	34	35	36	37	38
n_features	2.0	1.0	1.0	1.0	2.0	2.0	2.0	3.0	1.0	2.0	...	4.0	4.0	4.0	4.0	4.0	1.0	1.0	2.0	1.0	1.0

- Then, we get the smallest MSE with an optimal alpha for every window.

Macro Forecasting Example: Expanding Window Algorithm (Lasso)

We can obtain all the prediction values (testing data) and calculate the OOS R^2

```
> LassoExp_pred_data = []
> for i in range(table_pred.shape[1]):
    values_to_insert = table_pred.iloc[:, i].values[0]
    for value in values_to_insert:
        LassoExp_pred_data.append([value])
> LassoExp_pred_df = pd.DataFrame(LassoExp_pred_data)
LassoExp_pred_df:
```

0	0.002159
1	0.001580
2	0.002150
3	0.002162
4	0.002035
...	...
463	0.002219
464	0.002217
465	0.002228
466	0.002225
467	0.002215

Macro Forecasting Example: Expanding Window Algorithm (Lasso)

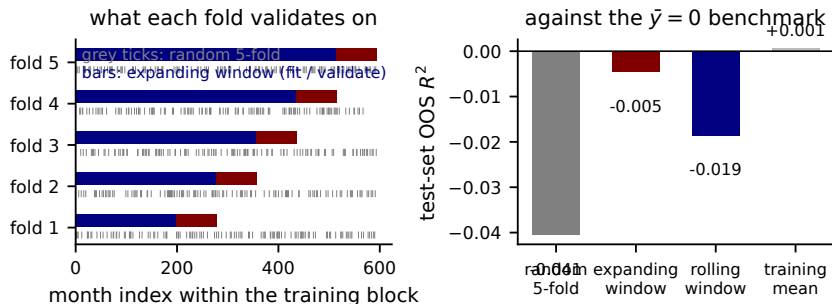
We can obtain all the prediction values (testing data) and calculate the OOS R^2

```
> test = y_full[initial_train_size + validation_size :  
               last_train_data + validation_size + test_size + 1]  
> trailing_mean = y_full[:last_train_data+validation_size+1].mean()  
> dev0 = deviance(test, trailing_mean, family = 'gaussian')  
> dev = deviance(test, LassoExp_pred_df.values, family = 'gaussian')  
LassoExp_R2 = 1-dev/dev0
```

LassoExp_R2: 0.01

- Again, you **must not use test data** when you construct the model.

What Does the Wrong Fold Actually Cost?



Same panel, same LASSO, same α grid. The *only* change is how the training block is cut into validation folds.

- **Random 5-fold:** $\alpha = 4.4 \times 10^{-4}$, 23 variables, test score **-0.0405**.
- **Expanding window:** $\alpha = 7.0 \times 10^{-4}$, 15 variables, **-0.0046**. Rolling: -0.0186.

The random fold sees the future, so validation error looks too good at small α and it **under-regularizes** — costing 0.036 of out-of-sample R^2 on a target whose entire signal is smaller than that.

Reporting an Out-of-Sample R^2

Three conventions, and you must say which you used:

- ▶ **What is the benchmark?** Above we used $\bar{y} = 0$ — forecast no growth surprise at all. A *test-block* mean is not knowable in advance, so it is not a legitimate benchmark for a forecast you claim to have made.
- ▶ **A negative R^2 is a result, not a bug.** It says the model is worse than the benchmark. Every model on the previous slide is negative. **Report it. Do not quietly switch benchmarks until it turns positive.**
- ▶ **How many things did you try?** If the reported model won a search, the size of the search belongs next to the number.

And one sanity check worth internalising: published *monthly* stock-level out-of-sample R^2 runs about 0.3–0.5%. Any far larger number on financial data is a prompt to go looking for the leak before celebrating.

One Thing to Keep Next to the Keyboard

Nearly every inflated out-of-sample number comes from one of a small number of mistakes, and they recur:

- ▶ a transformation fitted *before* the split — a scaler, an imputation, a PCA, a winsorisation cut-off;
- ▶ a merge keyed on a period-end rather than a publication date;
- ▶ a random fold on time-ordered data;
- ▶ a threshold or hyperparameter chosen on the test block;
- ▶ a benchmark quietly changed after the fact.

Keep this list. The five patterns above are the leakage questions that matter — ask them of every notebook, yours or a machine's, before you submit any homework or the final project. The first and the third are graded as fatal.

This is not a marking rubric. It is the list of things that will otherwise make you believe a result that is not there.

And a Warning About Borrowed Code

You are allowed to use ChatGPT, Claude or Copilot on everything in this course, and you should. But the two lines they most reliably write are the two this lecture just spent twenty minutes on.

- ▶ `train_test_split(X, y)` — its default is `shuffle=True`. On a time-ordered panel that is the random fold from three slides ago, and it cost 0.036 of out-of-sample R^2 . The third pattern on the last slide.
- ▶ `StandardScaler().fit_transform(X)` **before the split** — the scaler learns the mean and variance of rows you are about to hold out. The first pattern on the last slide. Lecture 7 measures this one.

Neither raises a warning. Both produce a number. That is why the last slide's list exists, and why you read code before you run it — whoever, or whatever, wrote it.

See [AI.Coding.Guide.pdf](#) on Canvas.

Ridge or LASSO?

Both are widely used machine learning techniques for linear models.

Both can be easily extended to non-linear models.

The choice depends on your assumptions about the data-generating process:

- ▶ LASSO:

- ▶ Best when you believe only a few features are important.
- ▶ Useful for feature selection and improved interpretability.

- ▶ Ridge:

- ▶ Ideal when most features are relevant, but you need to address multicollinearity or prevent overfitting.
- ▶ Suitable when features are weak and should still contribute to the model.